

Informe Técnico: Mutant Detector API

Examen Técnico MercadoLibre - Backend Developer

Autor: Leila Cruz

Fecha: 24 de Noviembre, 2025

Repositorio: <https://github.com/leila-cruz/ExamenMercadoLibreGlobal.git>

URL Base (API): <https://examenmercadolibreglobal.onrender.com>

Documentación Interactiva:
<https://examenmercadolibreglobal.onrender.com/swagger-ui/index.html>

1. Objetivo del Proyecto

El objetivo principal de este proyecto es desarrollar una API REST eficiente capaz de detectar si un humano es un "mutante" basándose en su secuencia de ADN. El sistema recibe una matriz de cadenas de ADN (NxN) y determina si existen **más de una secuencia de cuatro letras iguales** en dirección horizontal, vertical u oblicua.

Además de la lógica de detección, el sistema debe ser capaz de manejar altos volúmenes de tráfico, persistir los resultados para evitar re-procesamientos y ofrecer estadísticas en tiempo real.

2. Arquitectura y Tecnologías

El proyecto fue construido utilizando una arquitectura en capas para asegurar la separación de responsabilidades, mantenibilidad y escalabilidad.

- **Lenguaje:** Java 21 (LTS).
- **Framework:** Spring Boot 3.3.x (Web, Data JPA, Validation).
- **Persistencia:** H2 Database (Motor en memoria optimizado para lectura/escritura rápida).
- **Infraestructura:** Docker (Contenerización) y Render (Cloud Deployment).
- **Testing:** JUnit 5, Mockito y JaCoCo.

Estructura de Capas

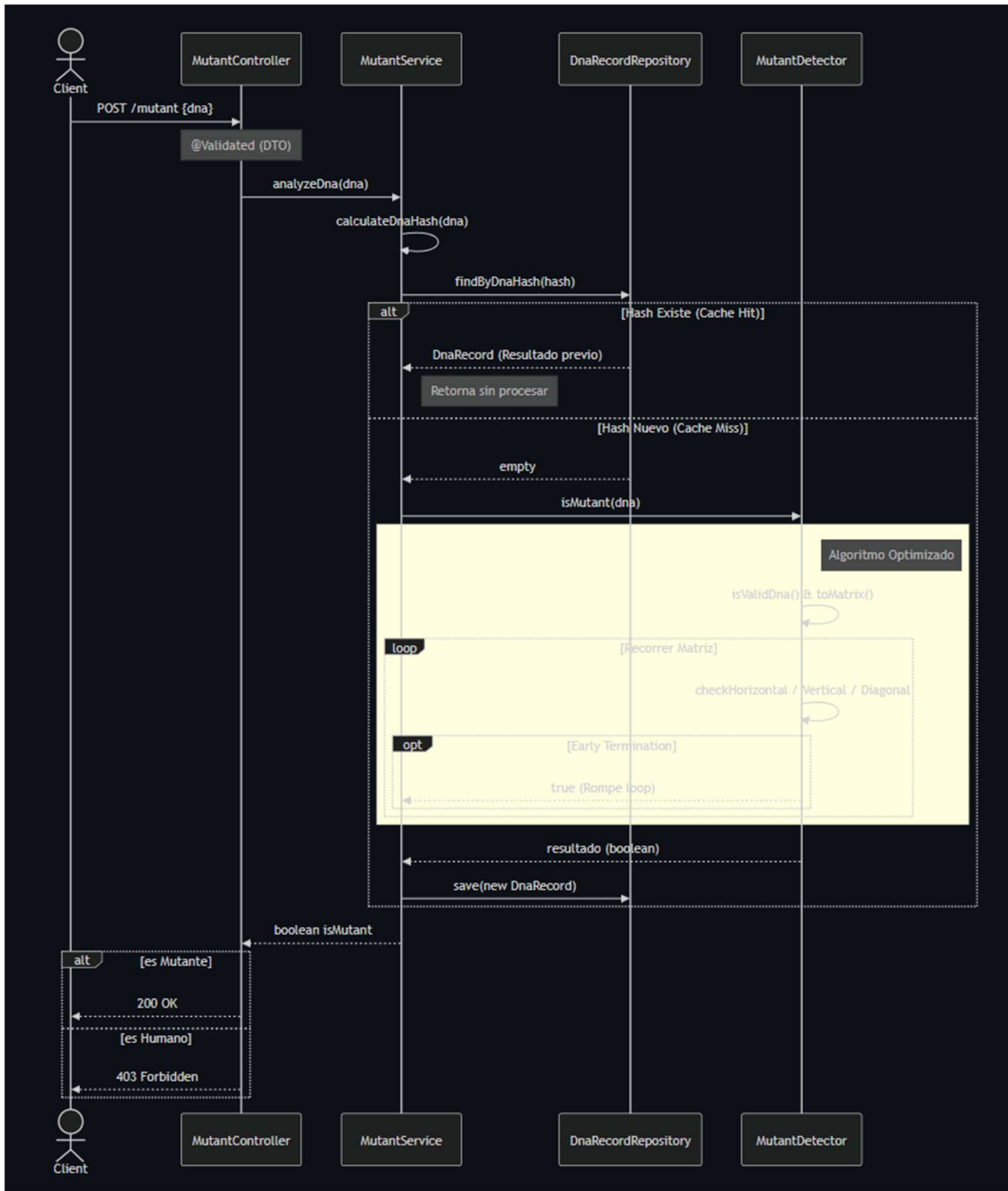
1. **Controller:** Maneja las peticiones HTTP y validaciones de entrada (DTOs).
2. **Service:** Contiene la lógica de negocio, orquestación y cálculo de Hashes.
3. **Repository:** Capa de acceso a datos utilizando Spring Data JPA.
4. **Component (Detector):** Algoritmo puro de detección optimizado.

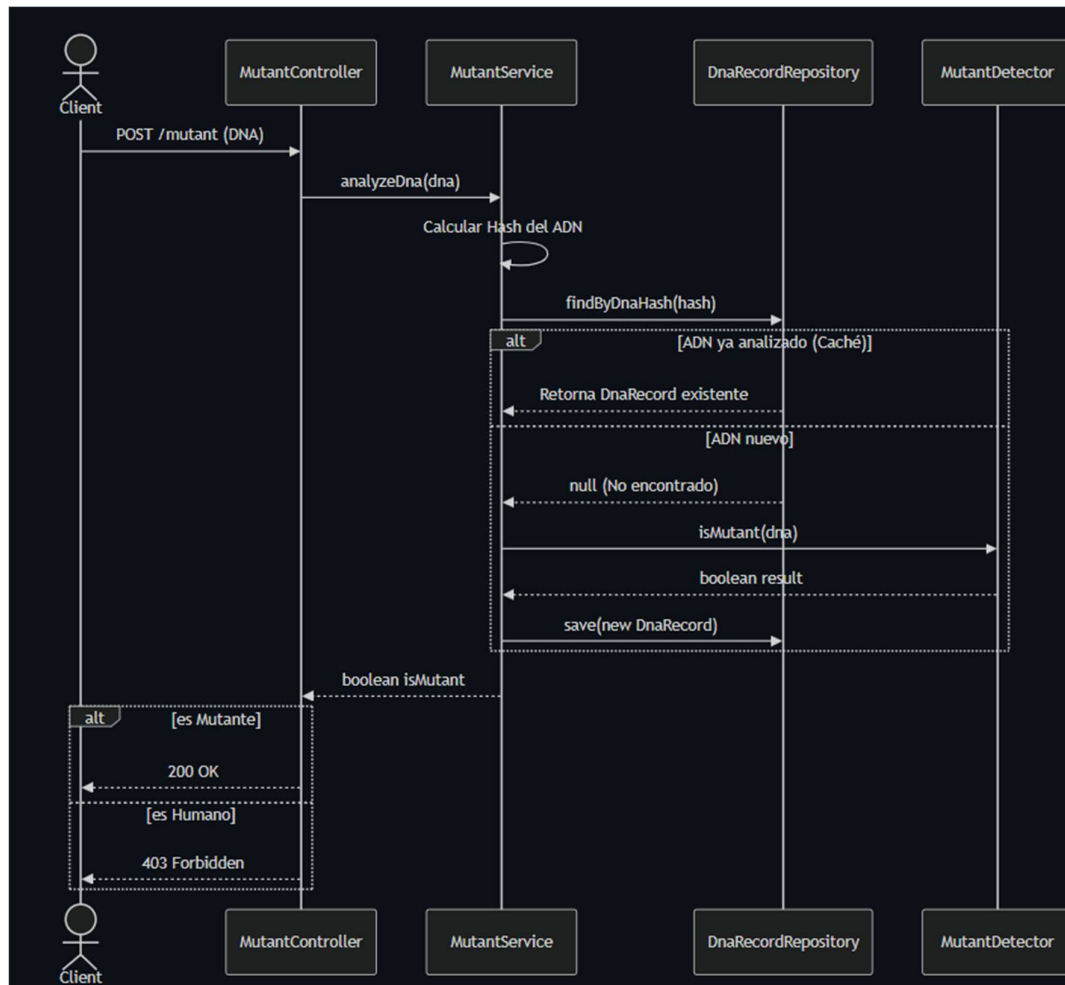
3. Diagramas de Secuencia (Nivel 3)

A continuación se detalla el flujo de interacción entre los componentes del sistema para los dos endpoints principales.

3.1. Detección de Mutantes (POST /mutant)

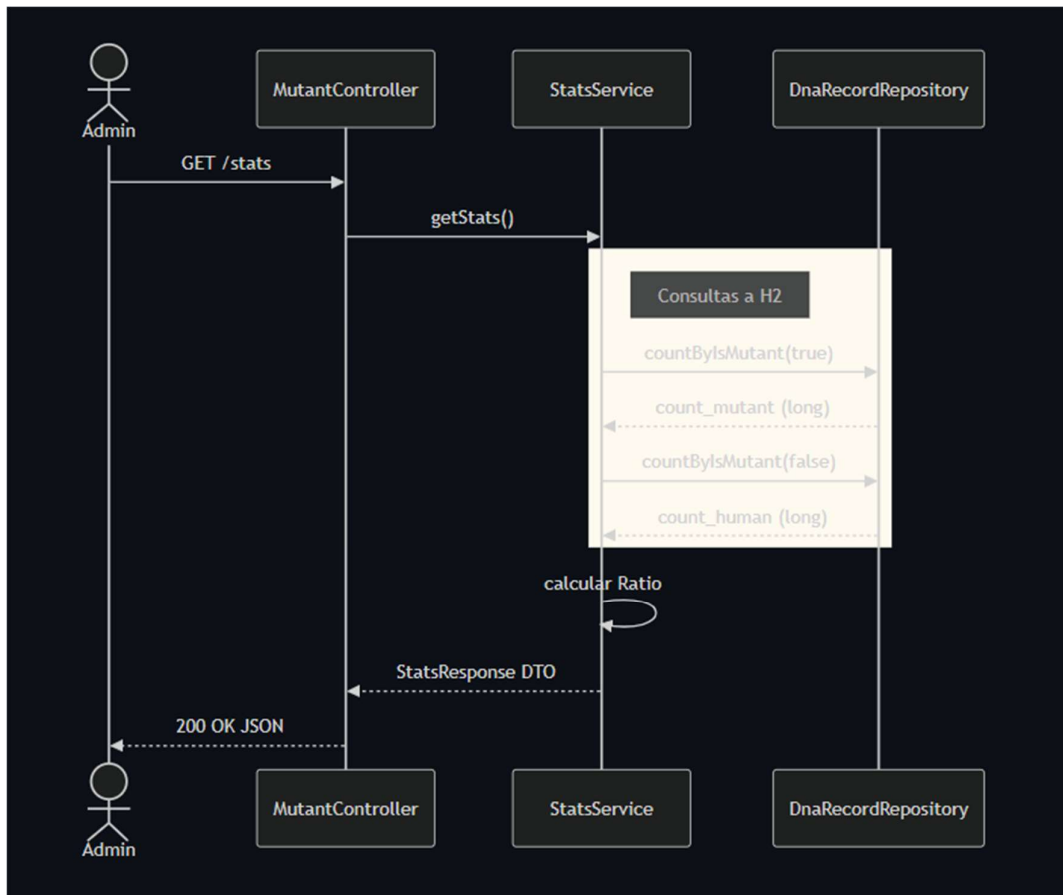
Este flujo incluye la optimización de caché: antes de procesar el ADN, se consulta a la base de datos si ese ADN ya fue analizado previamente utilizando un Hash SHA-256 único.





3.2. Consulta de Estadísticas (GET /stats)

Este flujo muestra cómo se recuperan los contadores de mutantes y humanos para calcular el ratio en tiempo real.



4. Estrategia de Persistencia y Datos

Para cumplir con el requerimiento del **Nivel 3** (Persistencia y Estadísticas), se implementó una estrategia centrada en la eficiencia y la integridad de datos.

Modelo de Datos (Entidad **DnaRecord**)

Se utiliza una base de datos relacional H2. La tabla principal `dna_records` tiene la siguiente estructura:

Campo	Tipo	Descripción
id	BIGINT (PK)	Identificador autoincremental.
dna_hash	VARCHAR(64)	Hash SHA-256 único de la secuencia de ADN.

is_mutant	BOOLEAN	Resultado del análisis (true/false).
created_at	TIMESTAMP	Fecha de verificación.

Optimización por Hashing

En lugar de almacenar la matriz completa de ADN (que podría ser muy grande, ej: 1000x1000), se genera un **Hash único** del contenido.

- **Beneficio:** Búsquedas $O(1)$ o $O(\log N)$ indexadas.
- **Restricción:** Se configuró una restricción UNIQUE en la columna dna_hash para garantizar que **solo exista 1 registro por ADN**, evitando duplicados y cálculos redundantes¹.

5. Escalabilidad y Manejo de Tráfico

Considerando que el sistema debe soportar alta concurrencia y fluctuaciones agresivas de tráfico², se implementaron las siguientes medidas de eficiencia:

1. Algoritmo con "Early Termination":

El detector no recorre la matriz completa innecesariamente. Apenas encuentra la segunda secuencia requerida para confirmar un mutante, detiene la ejecución y retorna true. Esto reduce drásticamente el uso de CPU en casos positivos.

2. Índices de Base de Datos:

Se crearon índices en las columnas dna_hash (para búsqueda rápida de duplicados) y is_mutant (para el cálculo instantáneo de estadísticas COUNT sin hacer un full table scan).

3. Stateless API:

La aplicación no guarda estado de sesión. Esto permite escalar horizontalmente en la nube (agregar más instancias de Docker en Render) detrás de un balanceador de carga sin romper la lógica del negocio.

4. Evitar Solapamientos (Lógica):

Se implementó lógica avanzada para asegurar que una secuencia de 5 letras iguales (AAAAA) cuente como una sola secuencia y no dos, evitando falsos positivos que podrían afectar la integridad de las estadísticas globales.

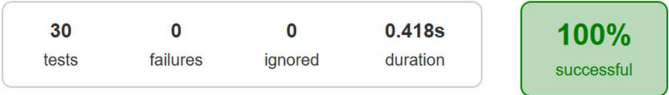
6. Calidad de Código y Testing

Se desarrolló una suite de pruebas exhaustiva cubriendo **Unit Testing**, **Integration Testing** y **Performance Testing**.

Reporte de Cobertura (JaCoCo)

El proyecto cumple y supera el requisito de cobertura de código (> 80%), asegurando que tanto la lógica del algoritmo como los controladores y entidades han sido probados.

Test Summary



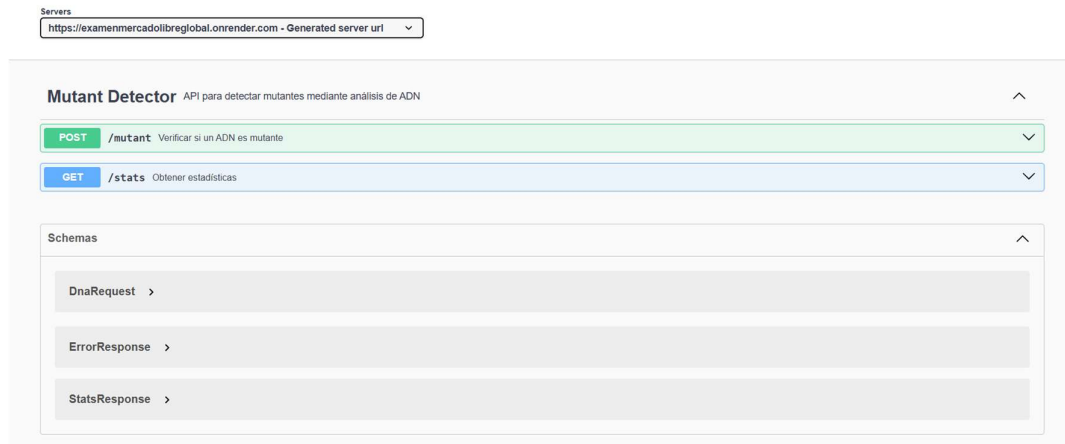
Packages		Classes			
Package	Tests	Failures	Ignored	Duration	Success rate
org.example.controller	1	0	0	0.043s	100%
org.example.dto	6	0	0	0.011s	100%
org.example.exception	4	0	0	0.215s	100%
org.example.service	19	0	0	0.149s	100%

Escenarios Probados

- Mutantes horizontales, verticales y diagonales.
- Humanos sin secuencias o con una sola.
- Matrices gigantes (1000x1000) para validar performance (< 500ms).
- Validación de entradas (Null, Empty, Caracteres inválidos).
- Manejo de excepciones y códigos HTTP correctos.

7. Documentación de la API

La API se encuentra documentada bajo el estándar **OpenAPI 3.0 (Swagger)**.



Se puede acceder a la documentación interactiva y probar los endpoints directamente desde el despliegue en la nube.